

Università degli Studi di Perugia

Dipartimento di Ingegneria

Corso di Laurea Triennale in

Ingegneria Informatica ed Elettronica

Tesi di Laurea

TLA+ per la verifica della correttezza del software

Laureando:

Federico Capitani

Relatore:

Prof. Luca Grilli

Anno Accademico 2024/2025

Indice

Introduzione **3**

1 Concetti fondamentali della verifica formale **6**

1.1 Origini e sviluppo dei metodi formali 7

1.2 Rappresentazione dei sistemi 8

1.3 Proprietà dei sistemi 9

1.4 Metodi formali nel ciclo di sviluppo: uso pratico, benefici e limiti 11

2 Il linguaggio TLA+ **14**

2.1 Origini e principi 14

2.2 Moduli, costanti e variabili 15

2.3 Struttura di una specifica 17

2.4 Azioni e transizioni di stato 19

2.5 Proprietà temporali 21

2.6 Il linguaggio PlusCal 23

2.7 Convenzioni e notazioni 24

3 Strumenti TLA+ **26**

3.1 Toolbox 26

3.2 TLC: il model checker 30

3.3 Apalache: verifica simbolica 32

3.4 TLAPS: sistema di dimostrazione formale 33

4 Processo di specifica e verifica 35

4.1 Dalla descrizione informale alla specifica formale 36

4.2 Modellazione incrementale e raffinamento 38

4.3 Verifica automatica del modello 40

4.4 Aggiornamento e integrazione 42

5 Casi di studio di TLA+ 44

5.1 Il protocollo Two-Phase Commit 45

5.2 Amazon S3 52

5.3 Altri casi di adozione industriale 55

Conclusioni 57

Bibliografia 59

Introduzione

Questo elaborato analizza il ruolo della verifica formale del software, con particolare attenzione al linguaggio *TLA+* (Temporal Logic of Actions), sviluppato da Leslie Lamport per descrivere in modo rigoroso e verificabile il comportamento dei sistemi concorrenti e distribuiti. Negli ultimi anni i sistemi informatici sono diventati sempre più complessi e diffusi anche in contesti critici, come i servizi cloud, la finanza o la gestione di dati sensibili. Per questo motivo è cresciuta la necessità di strumenti che aiutino a garantire la correttezza e la sicurezza del software. Tra queste, la verifica formale si distingue perché permette di descrivere matematicamente il comportamento di un sistema e di dimostrarne la conformità ai requisiti progettuali. TLA+ unisce logica temporale e teoria degli insiemi, offrendo una notazione formale rigorosa ma relativamente semplice da leggere anche per chi non è specialista. Esso consente di rappresentare l'evoluzione di un sistema come una sequenza di stati e transizioni e di verificare automaticamente, tramite strumenti dedicati, che determinate proprietà di sicurezza e correttezza siano sempre rispettate. Questo approccio permette di scoprire eventuali ambiguità o contraddizioni già nelle fasi iniziali di progetto, evitando che gli errori vengano scoperti solo dopo l'implementazione. Gli obiettivi principali dell'elaborato sono:

- evidenziare l'importanza della verifica formale nello sviluppo di applicazioni affidabili e prive di ambiguità logiche;

- analizzare il linguaggio TLA+ come mezzo per formalizzare e verificare la correttezza dei sistemi software complessi;
- illustrare le funzionalità offerte dagli strumenti di supporto, utili a tradurre la teoria in pratica operativa;
- mostrare, attraverso casi di studio concreti, come l'approccio formale consenta di individuare e prevenire errori di progettazione prima dell'implementazione.

Il testo è organizzato in modo progressivo e ogni capitolo affronta un aspetto specifico del tema, come descritto di seguito:

- Capitolo 1 *Concetti fondamentali della verifica formale* — presenta i principi teorici di base, come stati, transizioni e proprietà, che costituiscono il linguaggio comune per descrivere il comportamento dei sistemi.
- Capitolo 2 *Il linguaggio TLA+* — analizza in dettaglio la struttura del linguaggio di specifica, illustrandone i costrutti principali (moduli, variabili, azioni, formule temporali) e mostrando come vengano impiegati per modellare il comportamento dei sistemi concorrenti e distribuiti.
- Capitolo 3 *Gli strumenti di supporto* — descrive l'ambiente software utilizzato per la modellazione e la verifica, con particolare riferimento al Toolbox e al model checker TLC.
- Capitolo 4 *Processo di specifica e verifica* — ripercorre le fasi operative che conducono dalla definizione del modello alla validazione delle proprietà, evidenziandone il flusso logico.
- Capitolo 5 *Casi di studio* — analizza due esempi significativi di applicazione, il protocollo Two-Phase Commit e il sistema di replica di Amazon

S3, per mostrare come l'approccio formale aiuti a prevenire errori di progettazione.

- Capitolo 6 *Conclusioni* — sintetizza i risultati ottenuti e riflette sul ruolo della verifica formale come garanzia di affidabilità e sicurezza nei sistemi complessi.

Capitolo 1

Concetti fondamentali della verifica formale

I *metodi formali* sono tecniche di progettazione e verifica che si basano sulla matematica per descrivere e analizzare il comportamento dei sistemi software e hardware. A differenza dei test tradizionali, che controllano solo alcuni casi di esecuzione, i metodi formali permettono di ragionare in modo astratto su tutti i possibili comportamenti di un sistema, individuando errori anche prima della sua realizzazione.

L'idea principale è rappresentare il sistema attraverso un *modello formale*, cioè un insieme di regole e relazioni espresse in linguaggio logico, capace di definire con precisione quali stati il sistema può assumere e come può passare da uno stato all'altro. In questo modo la correttezza diventa una proprietà dimostrabile, non solo osservabile tramite test o simulazioni.

L'impiego dei metodi formali è particolarmente utile nei sistemi complessi o critici — come quelli aerospaziali, bancari o distribuiti — dove anche un piccolo errore può avere conseguenze gravi. In questi casi, la verifica matematica fornisce una copertura molto più ampia e affidabile dei possibili comportamenti

rispetto ai test empirici. Queste tecniche non sostituiscono completamente lo sviluppo tradizionale, ma lo affiancano, fornendo una base teorica solida su cui costruire il software finale.

1.1 Origini e sviluppo dei metodi formali

L'adozione di metodi matematici per descrivere il comportamento del software nasce dal bisogno di affrontare la crescente complessità dei programmi. Negli anni Settanta, l'aumento delle dimensioni e dell'interconnessione dei sistemi rese evidente che test e simulazioni non bastavano più: anche un software ampiamente collaudato continuava a produrre errori imprevisti. Da qui l'idea di utilizzare la logica come strumento per dimostrare la correttezza di un sistema già in fase di progettazione.

Le prime basi teoriche furono poste da Edsger Dijkstra e Tony Hoare. Il primo introdusse il concetto di *invariante* per esprimere le proprietà che un algoritmo deve mantenere vere durante la sua esecuzione; il secondo sviluppò la logica di Hoare, capace di formalizzare la relazione tra precondizioni e postcondizioni di un programma. Negli anni Ottanta, Leslie Lamport estese queste idee ai sistemi concorrenti e distribuiti introducendo la logica temporale, che consente di descrivere l'evoluzione dei sistemi nel tempo.

Nel tempo, i metodi formali hanno superato l'ambito accademico per entrare a far parte degli strumenti ingegneristici utilizzati nella progettazione e nella verifica dei sistemi. Sono stati sviluppati linguaggi e metodologie come Z, B-Method, CSP e TLA+, che uniscono rigore matematico e applicabilità concreta. L'obiettivo comune è quello di rendere la correttezza del sistema un aspetto verificabile fin dalle prime fasi dello sviluppo.

1.2 Rappresentazione dei sistemi

La rappresentazione di un sistema è il punto di partenza di ogni metodo formale. Per poter essere analizzato matematicamente, un sistema deve essere descritto in modo astratto, tramite un modello che ne catturi gli aspetti essenziali senza dipendere dai dettagli dell'implementazione.

Un modello formale descrive un sistema attraverso i suoi stati e i passaggi che li collegano. Uno stato rappresenta la condizione attuale del sistema, cioè i valori che le variabili assumono in un dato momento. Le transizioni indicano invece come e quando il sistema può cambiare stato, in risposta ad azioni, eventi o condizioni logiche. Nel complesso, il comportamento del sistema si può vedere come una successione ordinata di stati collegati tra loro da queste transizioni.

Questa visione a stati fornisce una base unificata per modellare sistemi di natura diversa:

- nei sistemi sequenziali, ogni transizione rappresenta l'esecuzione di un'istruzione o di un passo di calcolo;
- nei sistemi concorrenti, più transizioni possono essere possibili contemporaneamente, generando comportamenti alternativi;
- nei sistemi distribuiti, ciascun componente possiede un proprio stato locale e l'evoluzione complessiva è data dalla combinazione degli stati di tutti i componenti.

Per rendere il modello analizzabile, lo spazio degli stati deve essere finito oppure finitamente rappresentabile in relazione allo strumento adottato. Ciò richiede di scegliere un livello di astrazione adeguato:

si omettono dettagli non rilevanti e ci si concentra sulle variabili che influenzano le domande di interesse. Un buon modello formale non è una copia del

sistema reale, ma una semplificazione coerente, costruita per rispondere a quesiti precisi sulla correttezza o sul funzionamento; include inoltre le assunzioni sull'ambiente e i limiti di validità.

La descrizione del comportamento può usare notazioni diverse (diagrammi di stato, tabelle di transizione, formule logiche, reti di processi), ma l'obiettivo resta lo stesso: rendere esplicite le regole di evoluzione per poterle studiare in modo oggettivo. Tipicamente l'astrazione può essere sui dati (riduzione dei domini), sul tempo (discretizzazione o omissione), o composizionale (moduli con interfacce). Questa impostazione consente di applicare metodi di verifica automatica o deduttiva trasformando un problema ingegneristico in un problema logico ben definito.

1.3 Proprietà dei sistemi

Una volta costruito un modello che rappresenta il comportamento di un sistema, occorre definire le *proprietà* che esso deve rispettare. Le proprietà rappresentano i requisiti formali di correttezza e servono a verificare se il modello rispetta gli obiettivi funzionali e di affidabilità previsti. In pratica, esprimono ciò che il sistema deve sempre garantire e ciò che, prima o poi, deve accadere.

Dal punto di vista logico, una proprietà è una formula che impone restrizioni sugli stati o sulle sequenze di stati. Alcune proprietà si riferiscono a configurazioni statiche del sistema, altre al suo comportamento nel tempo. Per questo motivo si distinguono comunemente due categorie principali: *proprietà statiche* e *proprietà dinamiche*.

Le proprietà statiche descrivono condizioni locali, valide in un singolo stato. Servono, ad esempio, a verificare che i tipi di dato siano coerenti o che certe variabili rispettino vincoli di dominio. Sono utilizzate per garantire che il

modello non assuma configurazioni interne incoerenti o prive di senso rispetto alla logica del sistema.

Le proprietà dinamiche descrivono come il sistema si evolve nel tempo. Riguardano le relazioni tra stati successivi o intere sequenze di stati, specificando i comportamenti ammessi durante l'esecuzione. In questo contesto si distinguono due categorie fondamentali: le *proprietà di sicurezza* (safety) e le *proprietà di vivacità* (liveness).

Le proprietà di sicurezza impongono che nulla di scorretto o indesiderato possa accadere. Un sistema le soddisfa se, in qualunque esecuzione possibile, evita di entrare in uno stato che violi le condizioni di correttezza previste. Esempi classici sono la mutua esclusione in un protocollo di accesso a risorsa condivisa, l'assenza di overflow in un contatore o la coerenza dei dati in una memoria distribuita. In termini operativi, queste proprietà sono spesso espresse come *invarianti*, ossia predicati che devono rimanere veri in tutti gli stati raggiungibili. Se il verificatore trova una sequenza che porta alla loro violazione, produce una *traccia di controesempio* che mostra dove e come il comportamento si discosta dalle regole previste.

Le proprietà di vivacità, complementari alle precedenti, garantiscono che qualcosa di desiderato accada prima o poi. Non impediscono errori, ma assicurano che il sistema continui a progredire. Esempi tipici sono l'assenza di blocchi permanenti (deadlock), la garanzia che una richiesta riceva una risposta, o che un processo in attesa entri infine nella sezione critica. A differenza delle proprietà di sicurezza, non possono essere falsificate da un numero finito di passi: il mancato verificarsi di un evento potrebbe sempre avvenire in seguito. Per questo la loro verifica richiede meccanismi logici più complessi.

Nei sistemi concorrenti e distribuiti, dove più azioni possono essere possibili nello stesso istante, si introducono spesso condizioni di *fairness* (equità)

per rendere le proprietà di vivacità più realistiche. La fairness debole (*weak fairness*) stabilisce che, se un'azione rimane abilitata in modo continuo da un certo momento in poi, essa dovrà prima o poi essere eseguita. La fairness forte (*strong fairness*) impone che, se un'azione è abilitata infinite volte durante l'esecuzione, dovrà comunque verificarsi almeno una di esse. Queste condizioni non descrivono un comportamento specifico del sistema, ma escludono quelli "scorretti" in cui un'azione abilitata viene ignorata indefinitamente, garantendo così una nozione di progresso equo. Ad esempio, in un protocollo di accesso a risorsa condivisa, la *weak fairness* assicura che un processo abilitato in modo continuo non resti ignorato indefinitamente, mentre la *strong fairness* garantisce che un'azione che diventa periodicamente abilitata venga comunque eseguita almeno una volta.

Definire in modo chiaro le proprietà è un passaggio fondamentale nella modellazione formale. Un sistema può dirsi verificato solo rispetto alle proprietà che sono state esplicitamente dichiarate; se queste risultano vaghe o incomplete, la verifica perde di significato. È per questo che la formulazione dei vincoli logici richiede un equilibrio tra il rigore matematico e la comprensione del contesto in cui il sistema opera. Le nozioni di *safety*, *liveness* e *fairness* offrono un linguaggio condiviso per esprimere e controllare la correttezza, e saranno riprese nei capitoli successivi per analizzarne la traduzione in TLA+.

1.4 Metodi formali nel ciclo di sviluppo: uso pratico, benefici e limiti

I metodi formali non sostituiscono analisi, test o revisione del codice: si integrano nel processo e ne rafforzano l'affidabilità. Il loro contributo è massimo quando vengono applicati in modo mirato, sui componenti e sui requisiti più

critici. Dal punto di vista pratico, la connessione con l'ingegneria del software si manifesta in tre momenti principali:

- nelle fasi iniziali di progetto, per chiarire i requisiti ed eliminare ambiguità tramite una specifica formale orientata alle proprietà;
- durante l'architettura e la progettazione, per validare scelte di concorrenza, sincronizzazione e gestione di risorse;
- a supporto del collaudo, per individuare casi limite che i test empirici tendono a trascurare e per generare scenari di prova più mirati.

L'adozione efficace richiede alcune scelte consapevoli di modellazione:

- selezionare il livello di astrazione adeguato, limitando i dettagli al necessario per verificare le proprietà di interesse;
- esplicitare le assunzioni sull'ambiente (ingressi, guasti, tempi), così da delimitare il perimetro di validità del modello;
- formulare proprietà chiare e verificabili, distinguendo tra vincoli di sicurezza e obiettivi di vivacità.

I benefici principali sono la riduzione delle ambiguità, l'emersione precoce di errori di progettazione e la possibilità di ragionare su tutti i comportamenti ammessi dal modello, non solo su campioni di esecuzione.

Tali vantaggi hanno un costo: la costruzione del modello richiede tempo e competenze specifiche; inoltre, modelli troppo dettagliati possono portare a uno spazio degli stati difficile da gestire. Per questo, nella pratica, si privilegia un uso selettivo e incrementale, partendo dai sottosistemi più critici o dalle proprietà più rilevanti.

Un percorso di adozione realistico prevede passi piccoli e misurabili:

- introdurre invarianti semplici già in fase di analisi dei requisiti;
- modellare e verificare per primi i meccanismi di coordinamento e accesso a risorse condivise;
- raffinare il modello solo quando necessario per supportare nuove proprietà o scenari.

Questa collocazione nel ciclo di sviluppo consente di trarre valore dai metodi formali senza appesantire inutilmente il processo. Nel capitolo successivo verrà presentato un linguaggio specifico per la modellazione e la verifica dei comportamenti, che permette di applicare in modo concreto i principi introdotti fin qui.

Capitolo 2

Il linguaggio TLA+

TLA+ (Temporal Logic of Actions) è un linguaggio di specifica formale ideato da Leslie Lamport all'inizio degli anni Novanta. Concepito per descrivere e verificare in modo rigoroso il comportamento dei sistemi concorrenti e distribuiti, TLA+ fornisce ai progettisti uno strumento matematico capace di individuare errori di progettazione prima ancora dell'implementazione del codice. Il linguaggio combina logica temporale e teoria degli insiemi, permettendo di rappresentare l'evoluzione di un sistema come una sequenza di stati e azioni.

2.1 Origini e principi

TLA+ unisce la *logica temporale* alla descrizione delle *azioni di sistema*. Un sistema viene modellato come un insieme di stati e di transizioni tra di essi, definite da formule logiche che ne descrivono l'evoluzione nel tempo. A differenza delle specifiche informali in linguaggio naturale, le formule di TLA+ hanno una semantica precisa, fondata sulla teoria degli insiemi e sui predicati, e non lasciano spazio ad ambiguità interpretative.



Figura 2.1: Logo di TLA+.

Alla base del formalismo vi è la teoria degli insiemi di Zermelo–Fraenkel, estesa con operatori temporali che permettono di ragionare sull'ordine delle azioni nel tempo. Questo approccio, basato sul concetto di macchina a stati, consente di descrivere con precisione l'insieme dei comportamenti corretti che un sistema può assumere.

TLA+ funge da *progetto architettonico* per i sistemi software: non descrive come realizzarli, ma cosa devono fare, specificando le condizioni che governano l'evoluzione dello stato. Grazie alla sua base logico–matematica, permette di formulare e dimostrare proprietà di correttezza con rigore formale.

Il linguaggio è stato introdotto ufficialmente nel 1999, dopo anni di ricerca sui metodi di verifica formale, ed è stato adottato anche in ambito industriale — tra cui Amazon, Microsoft e Intel — come strumento di validazione per sistemi critici e distribuiti, dove correttezza e affidabilità sono requisiti essenziali.

2.2 Moduli, costanti e variabili

Una specifica TLA+ è organizzata in uno o più *moduli*. Ogni modulo rappresenta l'unità logica di base del linguaggio: contiene le definizioni, le formule e

le dichiarazioni che descrivono il comportamento del sistema, in modo analogo a un file sorgente che raggruppa il codice di un programma. La definizione di un modulo inizia con la parola chiave `-- MODULE NomeDelModulo --`, prosegue con il corpo del modulo e termina con la riga `====`. All'interno vengono dichiarate le costanti e le variabili utilizzate nella specifica.

```
------------------- MODULE Esempio -------------------
CONSTANTS N
VARIABLES x, y
...
=======================================================
```

Le *costanti* rappresentano valori fissi. Servono a definire parametri di configurazione, dimensioni o limiti numerici e vengono stabilite una sola volta, senza variare durante l'evoluzione del sistema. Il loro valore concreto può essere assegnato successivamente, nella fase di verifica del modello, senza modificare il testo della specifica. In questo modo le costanti funzionano come parametri generici che rendono la descrizione più astratta e riutilizzabile: ad esempio, una costante `N` può rappresentare un numero arbitrario di processi, assegnato poi a un valore preciso al momento dell'analisi.

Le *variabili di stato* descrivono invece la parte dinamica del sistema e possono assumere valori diversi nei vari stati. Una configurazione è definita dall'insieme dei valori correnti di tutte le variabili dichiarate nel modulo. Ad esempio, la dichiarazione `VARIABLES x, y` introduce due variabili di stato, `x` e `y`, i cui valori evolvono nel tempo tramite le transizioni di stato.

In TLA+ tutte le variabili sono globali al modulo: non esiste un concetto di variabile locale, a meno che non venga modellata esplicitamente come componente dello stato globale. Per convenzione si usano nomi minuscoli e descrittivi per le variabili (`clock`, `counter`, `buffer`), mentre costanti e insiemi

vengono scritti in maiuscolo (`N`, `KEY`, `VAL`), così da distinguerli visivamente e favorire la leggibilità.

In sintesi, le costanti rimangono immutate nel tempo, mentre le variabili rappresentano lo stato del sistema e ne descrivono l'evoluzione da uno stato al successivo.

2.3 Struttura di una specifica

In TLA+, una specifica completa è definita da una formula che vincola tutti i comportamenti ammessi del sistema. Tradizionalmente questa formula, denominata `Spec`, combina tre elementi principali:

- la condizione iniziale, che descrive l'insieme degli stati di partenza;
- la relazione di *next-state*, che stabilisce come il sistema evolve da uno stato al successivo;
- la formula temporale completa, che impone che ogni comportamento inizi in uno stato valido e che tutti i passi successivi rispettino la relazione di transizione.

La formula `Init` è un predicato di stato che descrive le condizioni iniziali del sistema:

```
Init == x = 0
```

oppure in forma più generale:

```
Init == x \in 0..10
```

`Init` individua quindi tutti gli stati validi di partenza, specificando quali valori possono assumere le variabili iniziali.

Un aspetto essenziale della sintassi è la distinzione tra variabili *primate* e *non primate*. Una variabile senza apice (ad esempio `x`) rappresenta il suo valore nello stato attuale, mentre la stessa variabile con l'apice (`x'`) denota il valore nello stato successivo. La formula `x' = x + 1` stabilisce quindi una relazione logica tra due stati, non un'istruzione di modifica.

Il passo successivo consiste nel definire la relazione di transizione, comunemente chiamata `Next`, che descrive le modifiche di stato possibili:

```
Next == x' = x + 1
```

Nella pratica, `Next` può essere composto da più sotto-azioni combinate in disgiunzione, ciascuna corrispondente a un possibile tipo di transizione (vedi sezione 2.4).

La formula completa della specifica è:

```
Spec == Init /\ [][Next]_<<vars>>
```

dove `«vars»` indica la tupla di tutte le variabili di stato del modulo (ad esempio `«x, y»`). L'operatore temporale `[]` impone che la relazione di transizione valga per tutti i passi di esecuzione. Il costrutto `[Next]_«vars»` permette i cosiddetti *passi di stuttering*, cioè transizioni in cui lo stato del sistema non cambia:

```
Next \/ (<<vars>>' = <<vars>>)
```

Significa: "a ogni passo o vale l'azione `Next`, oppure le variabili rimangono invariate". Questo meccanismo consente di rappresentare naturalmente i momenti di inattività nei sistemi concorrenti.

Talvolta si aggiungono vincoli di *fairness*, che assicurano che determinate azioni non vengano rimandate indefinitamente. Tali condizioni, utili per garantire proprietà di vivacità, saranno approfondite nella sezione 2.5.

In sintesi, una specifica TLA+ completa è costruita secondo la seguente struttura logica:

1. Definizione di costanti e variabili;
2. Predicato `Init` (stato iniziale);
3. Azioni che compongono `Next`;
4. Formula temporale `Spec`, che unisce le due componenti precedenti;
5. Eventuali proprietà aggiuntive, come invarianti o condizioni di fairness.

Il frammento seguente mostra un semplice esempio di specifica TLA+ per un contatore che si incrementa a ogni passo:

```
---------------------- MODULE Counter ----------------------
VARIABLE x

Init == x = 0
Next == x' = x + 1
Spec == Init /\ [][Next]_<<x>>
=============================================================
```

2.4 Azioni e transizioni di stato

Un'*azione* è una formula logica che descrive una transizione tra uno stato corrente e uno successivo. Tecnicamente è un predicato che coinvolge variabili riferite sia allo stato presente (non primate) sia a quello futuro (primate). Considerando una coppia di stati consecutivi `(s, s')`, dire che un'azione `A` è verificata significa che, sostituendo in `A` i valori delle variabili negli stati `s` e `s'`, la formula risulta vera.

```
AzioneInc == x' = x + 1
```

Se nel nuovo stato il valore di `x` è maggiore di uno rispetto a quello precedente, la formula è vera; altrimenti la transizione non soddisfa l'azione definita.

Nelle specifiche reali la relazione di transizione `Next` è spesso costituita da più azioni alternative, combinate mediante disgiunzione:

```
AzioneInc == x < Max /\ x' = x + 1
AzioneReset == x = Max /\ x' = 0
Next == AzioneInc \/ AzioneReset
```

Questa definizione descrive un contatore modulare che, a ogni passo, o incrementa `x` se è minore di `Max`, oppure la riporta a 0 quando raggiunge il valore massimo. Le due sotto-azioni sono mutuamente esclusive. Quando più azioni sono abilitate contemporaneamente, la disgiunzione introduce *non-determinismo*, poiché il sistema può compiere una qualsiasi delle transizioni consentite.

Se un'azione non menziona esplicitamente una variabile, il suo valore futuro sarebbe indeterminato. Per specificare che una variabile non cambia durante una transizione, TLA+ fornisce il costrutto `UNCHANGED`:

```
AzioneX == (x < 10) /\ x' = x + 1 /\ UNCHANGED y
```

La clausola `UNCHANGED y` equivale a `y' = y`. Nelle specifiche complesse si usa spesso `UNCHANGED «v1, v2, ...»` per elencare tutte le variabili costanti in quella transizione, migliorando la leggibilità e riducendo ambiguità.

In sintesi, il comportamento complessivo del sistema è rappresentato da una sequenza (potenzialmente infinita) di stati s_0, s_1, s_2, ... che modellano l'evoluzione del sistema nel tempo con semantica precisa e verificabile.

2.5 Proprietà temporali

Come introdotto nel Capitolo 1, le proprietà temporali descrivono l'evoluzione del sistema nel tempo, distinguendo ciò che deve sempre rimanere vero da ciò che deve prima o poi accadere. In TLA+ questi concetti vengono formalizzati attraverso formule logiche integrabili nella specifica, che gli strumenti di verifica possono analizzare direttamente.

Le proprietà di *safety* garantiscono che un comportamento scorretto non si verifichi mai. In TLA+ sono espresse come *invarianti*, ossia predicati di stato che devono risultare veri in tutti gli stati raggiungibili. Ogni invariante è dichiarato come una formula logica separata e viene poi collegato alla specifica complessiva mediante l'implicazione temporale:

```
Spec => []Inv
```

dove `Inv` rappresenta la condizione che deve valere in ogni passo dell'esecuzione. Ad esempio, in un contatore che non deve mai assumere valori negativi:

```
Inv == x >= 0
```

Durante la verifica, il model checker TLC (sezione 3.2) esplora lo spazio degli stati generato da `Spec` e controlla automaticamente che l'invariante non venga mai violato. Se trova una violazione, produce un controesempio, ossia una sequenza di stati che porta alla condizione errata. Questo meccanismo rende le proprietà di sicurezza non solo espressive, ma anche verificabili in modo sistematico e automatico.

Le proprietà di *liveness* completano il quadro assicurando che il sistema non resti bloccato in stati stazionari: qualcosa di desiderato deve prima o poi accadere. Mentre la sicurezza impedisce comportamenti errati, la vivacità

impone il verificarsi di eventi positivi. Tali vincoli si esprimono tramite l'operatore temporale `<>`, spesso combinato con `[]` per formulare schemi di tipo "richiesta–risposta":

```
[](Richiesta -> <>Concessione)
```

La formula afferma che ogni volta che viene generata una richiesta, essa sarà prima o poi seguita da una concessione. Questo schema è alla base di molti protocolli concorrenti e di sincronizzazione, in cui si vuole garantire che una risorsa condivisa venga infine concessa a tutti i processi che la richiedono.

A differenza delle proprietà di sicurezza, le violazioni di vivacità non possono essere rilevate da un numero finito di passi: un comportamento che appare bloccato potrebbe ancora evolvere. Per questo la loro verifica richiede l'introduzione di *condizioni di fairness*, che aggiungono ipotesi sul comportamento dell'esecuzione.

In TLA+ esistono due forme principali di fairness, applicate alle azioni definite in `Next`:

- `WF_vars(A)` — *weak fairness*: se l'azione `A` rimane continuamente abilitata a partire da un certo punto, essa dovrà prima o poi essere eseguita;
- `SF_vars(A)` — *strong fairness*: se l'azione `A` è abilitata infinite volte durante l'esecuzione, dovrà comunque verificarsi almeno una di esse.

Queste condizioni vengono aggiunte direttamente alla formula della specifica, ad esempio:

```
Spec == Init /\ [][Next]_<<vars>> /\ WF_vars(A)
```

La combinazione di operatori temporali e fairness consente di rappresentare in modo rigoroso il concetto di "progresso inevitabile", essenziale nei sistemi distribuiti.

Insieme, *safety*, *liveness* e *fairness* costituiscono la base del controllo formale dei comportamenti in TLA+ e saranno riprese nei capitoli successivi, dove se ne mostrerà la verifica pratica tramite gli strumenti dedicati.

2.6 Il linguaggio PlusCal

Nel processo di scrittura di specifiche TLA+, può risultare complesso descrivere direttamente il flusso di controllo di un algoritmo, specialmente quando si utilizzano strutture iterative o processi concorrenti. Per semplificare questa fase, Leslie Lamport ha introdotto un linguaggio di alto livello, concepito come pseudocodice formale tradotto automaticamente in TLA+. PlusCal unisce la familiarità della programmazione imperativa con la precisione logico-matematica di TLA+, permettendo di descrivere algoritmi in modo più intuitivo ma con la stessa semantica formale.

A differenza di TLA+, che si basa su formule e relazioni tra stati, PlusCal adotta una sintassi più vicina ai linguaggi di programmazione tradizionali: variabili locali, assegnamenti (`:=`), costrutti condizionali (`if/then/else`), cicli (`while`) e dichiarazioni di processi paralleli. Ogni algoritmo PlusCal è inserito in un modulo TLA+ come commento speciale, delimitato da `(*-algorithm ... *)`.

```
------------------------- MODULE Counter -------------------------
EXTENDS Naturals

(*--algorithm Contatore
variables n = 0;
begin
  while (n < 5) do
```

```
    n := n + 1;
  end while;
end algorithm; *)
\* ================================================================
```

L'algoritmo `Contatore` dichiara una variabile `n` inizializzata a 0 e la incrementa finché il suo valore è minore di 5. Il traduttore PlusCal produce automaticamente una specifica TLA+ equivalente, definendo `Init`, `Next` e `Spec`. L'istruzione `EXTENDS Naturals` importa dal modulo standard gli operatori aritmetici e di confronto.

PlusCal risulta particolarmente adatto a descrivere algoritmi con un flusso di controllo ben definito, come cicli o protocolli organizzati in fasi successive. Consente di esprimere in modo chiaro le relazioni tra i vari passi di esecuzione, mantenendo al tempo stesso un legame diretto con la logica formale di TLA+. In questo senso, funge da collegamento tra la scrittura algoritmica tradizionale e la specifica formale, semplificando il passaggio dalla progettazione alla fase di verifica automatica.

2.7 Convenzioni e notazioni

Nel seguito si riporta una tabella con i principali simboli matematici e la relativa notazione ASCII utilizzata in TLA+, per facilitarne la lettura nei frammenti di codice.

Simbolo	In TLA+	Significato
$\in$	`\in`	appartiene a
$\notin$	`\notin`	non appartiene a
$\cup$	`\cup`	unione
$\cap$	`\cap`	intersezione
$\setminus$	`\`	differenza insiemistica
$\emptyset$	`{}`	insieme vuoto
$\forall$	`\A`	quantificatore universale (per ogni)
$\exists$	`\E`	quantificatore esistenziale (esiste)
$=$	`=`	uguaglianza
$\neq$	`/=`	disuguaglianza
$\neg$	`~`	negazione logica
$\wedge$	`/\`	congiunzione (AND)
$\vee$	`\/`	disgiunzione (OR)
$\Rightarrow$	`=>`	implicazione
$\Leftrightarrow$	`<=>`	equivalenza logica
$\Box$	`[]`	operatore “sempre”
$\Diamond$	`<>`	operatore “eventualmente”
$Beh(S)$	`Beh(S)`	insieme dei comportamenti della specifica S

Tabella 2.1: Simboli matematici e sintassi corrispondente in TLA+.

Capitolo 3

Strumenti TLA+

In questo capitolo introdurremo, uno per uno, gli strumenti che compongono l'ambiente TLA+. Essi rappresentano l'estensione operativa del linguaggio, rendendo possibile passare dalla scrittura delle specifiche alla loro verifica automatica e, se necessario, alla dimostrazione formale delle proprietà.

Vedremo come `Toolbox` offra un ambiente di sviluppo integrato, come il model checker `TLC` consenta di esplorare lo spazio degli stati del sistema, e come strumenti avanzati come `Apalache` e `TLAPS` amplino le possibilità di analisi, introducendo rispettivamente la verifica simbolica e quella deduttiva.

L'obiettivo è mostrare come ciascuno di essi contribuisca, con un ruolo specifico, a rendere il metodo TLA+ completo e applicabile nella pratica.

3.1 Toolbox

Il *TLA+ Toolbox* è l'ambiente di sviluppo ufficiale per la scrittura e la verifica di specifiche. Sviluppato da Leslie Lamport in collaborazione con Microsoft Research, fornisce un'interfaccia grafica integrata (IDE) che semplifica l'intero

ciclo di lavoro: dalla redazione dei moduli alla definizione dei modelli di verifica con il model checker `TLC`.

Il Toolbox nasce con l'idea di rendere l'uso dei metodi formali più accessibile, anche a chi non ha dimestichezza con la riga di comando. Riunisce in un'unica interfaccia strumenti per l'editing, l'analisi sintattica, la configurazione e la verifica, offrendo così un ambiente completo e coerente per lavorare con TLA+.

Dal punto di vista strutturale, l'interfaccia è organizzata in tre aree principali:

- l'area dei moduli, dove si gestiscono i file `.tla` e `.cfg`, corrispondenti rispettivamente alle specifiche e ai modelli di verifica;
- l'editor, che evidenzia la sintassi del linguaggio, supporta il completamento automatico e fornisce collegamenti diretti alle definizioni di formule e operatori;
- il pannello dei risultati, che mostra lo stato della verifica, le violazioni individuate e i dettagli delle tracce di errore.

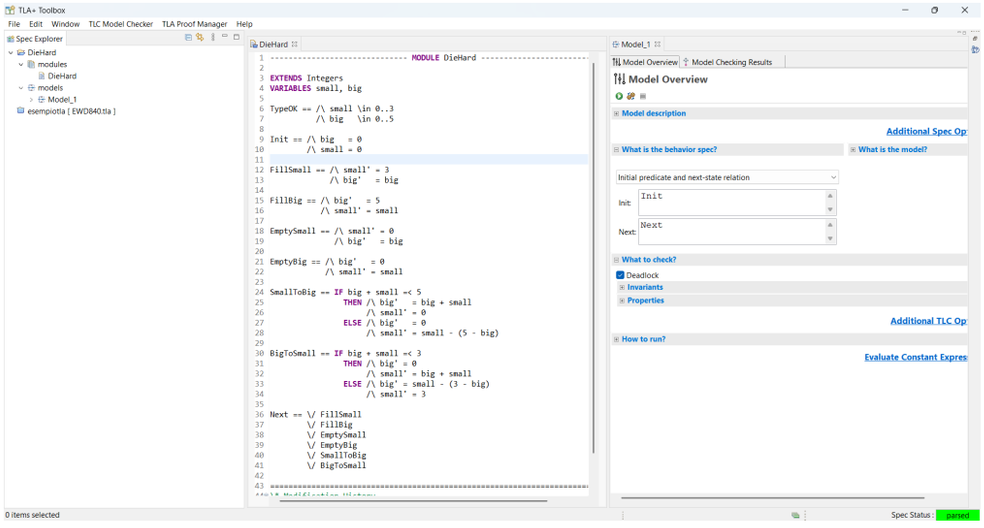


Figura 3.1: Interfaccia del *TLA+ Toolbox* con il modulo `DieHard` e la configurazione del model checker `TLC`.

Ogni progetto del Toolbox corrisponde a una specifica TLA+ e può contenere uno o più modelli di verifica, ciascuno definito da un file di configurazione con estensione `.cfg`. Il file di configurazione stabilisce i parametri del modello: valori assegnati alle costanti, invarianti da controllare, proprietà temporali da verificare e opzioni di esecuzione di `TLC`. Questo approccio consente di analizzare diversi scenari di uno stesso sistema senza modificare la specifica formale.

La verifica avviene tramite il comando `Run Model`, che avvia automaticamente `TLC` sul modello selezionato. Nel corso dell'analisi, il Toolbox fornisce un log dettagliato dello stato della verifica, mostrando il numero di stati esplorati, la profondità raggiunta e l'eventuale individuazione di controesempi. Quando una proprietà risulta violata, il sistema genera una *error trace*, una sequenza di stati che illustra passo per passo come il comportamento del sistema porta alla violazione. Questa funzionalità è particolarmente utile nella fase di *debugging* dei modelli.

Il Toolbox integra inoltre il traduttore PlusCal, che consente di scrivere

un algoritmo in pseudocodice e tradurlo automaticamente in TLA+. Il risultato della traduzione viene visualizzato in un modulo separato, mantenendo la corrispondenza con le formule TLA+. In questo modo l'ambiente offre un passaggio immediato dal livello algoritmico a quello formale.

Oltre alle sue funzioni principali, il Toolbox offre una serie di strumenti aggiuntivi che ne estendono le capacità operative:

- il salvataggio automatico e la gestione delle versioni dei moduli, utili per tenere traccia delle modifiche e ripristinare versioni precedenti;
- la possibilità di importare moduli esterni tramite l'istruzione `EXTENDS`, che favorisce il riuso delle specifiche e la modularità;
- l'integrazione con sistemi di versionamento e strumenti di analisi statica, particolarmente utile in ambienti di lavoro condivisi o industriali;
- un visualizzatore di trace interattivo, che consente di esaminare passo passo l'evoluzione dello stato del sistema e i cambiamenti delle variabili;
- la configurazione di opzioni avanzate per `TLC`, come la profondità di esplorazione, il numero di thread o i limiti di memoria;
- l'esportazione dei risultati della verifica in formato testuale, XML o grafico, per documentare le analisi o integrarle in report esterni.

Queste funzionalità rendono il Toolbox uno strumento completo non solo per la scrittura di specifiche, ma anche per la loro sperimentazione e validazione. Riunisce in un'unica interfaccia tutte le fasi del processo di modellazione formale.

3.2 TLC: il model checker

`TLC` è il *model checker* ufficiale di TLA+. A partire da una specifica definita dalle formule `Init`, `Next` e, se presente, `Spec`, esplora in modo sistematico lo spazio degli stati generabili dal sistema e verifica automaticamente che le proprietà dichiarate siano rispettate. Implementato in linguaggio Java e distribuito come progetto *open source*, è incluso nell'ambiente `Toolbox` e può essere utilizzato anche separatamente. Il suo funzionamento si basa sull'assunzione che le costanti della specifica abbiano valori finiti, così da rendere lo spazio degli stati esplorabile in modo esaustivo o, in alternativa, simulabile.

Il processo di verifica inizia con la valutazione della formula `Init`, che definisce l'insieme degli stati iniziali possibili. Per ciascuno di essi, `TLC` applica la relazione di transizione `Next`, generando tutti i successori e costruendo progressivamente il grafo degli stati raggiungibili. L'esplorazione avviene in ampiezza (*breadth-first search*), in modo da individuare rapidamente controesempi di lunghezza minima. A ogni stato è associata un'impronta digitale, o *fingerprint*, calcolata tramite funzioni di hashing: ciò consente di evitare la riesplorazione di configurazioni già visitate e di ridurre l'uso di memoria.

Oltre alla modalità di verifica completa, `TLC` può operare in *modalità di simulazione*. In questo caso genera casualmente sequenze di stati conformi alle regole di transizione, utile per individuare errori evidenti senza esplorare tutto lo spazio. Entrambe le modalità possono essere parallelizzate: il motore di verifica distribuisce l'analisi su più thread, ciascuno incaricato di espandere una parte distinta del grafo.

Durante la verifica, `TLC` valuta automaticamente tutte le proprietà dichiarate nel modello. Le proprietà di sicurezza vengono controllate sotto forma di invarianti: se un invariante risulta falso in uno stato, il verificatore interrompe l'esecuzione e genera una *traccia di errore*, ossia la sequenza di stati che porta

alla violazione. Il controllo di *deadlock* è anch'esso automatico e consiste nel verificare che da ogni stato sia abilitata almeno una transizione valida; in caso contrario, lo stato viene segnalato come bloccato.

Per le proprietà temporali, `TLC` analizza la presenza di cicli nel grafo che possano costituire una violazione della formula. Un esempio tipico è il rilevamento di una sequenza ciclica in cui una condizione di richiesta (`Request`) resta indefinitamente vera senza che la corrispondente concessione (`Grant`) si verifichi, nonostante le condizioni di fairness. Le ipotesi di equità, definite tramite gli operatori `WF_vars(A)` o `SF_vars(A)`, permettono di limitare i casi in cui un'azione abilitata venga ignorata all'infinito, garantendo che le proprietà di vivacità possano essere verificate correttamente.

Per gestire la crescita del dominio esplorato, `TLC` include diverse tecniche di riduzione. Tra le più efficaci vi è la *riduzione per simmetria*, che consente di trattare come equivalenti stati identici a meno di una permutazione di elementi indistinguibili (ad esempio processi con comportamento identico). Altri strumenti di controllo sono i *vincoli di stato* e di *azione*, che permettono di limitare l'esplorazione a una porzione significativa del comportamento del sistema, escludendo configurazioni irrilevanti ai fini della verifica. Durante l'analisi, il verificatore può inoltre creare *checkpoint* su disco per sospendere e riprendere l'esecuzione senza perdita di dati.

Il comportamento di `TLC` è configurato tramite un file con estensione `.cfg`, che definisce i parametri di esecuzione e i valori concreti delle costanti. Un esempio di configurazione è:

```
SPECIFICATION Spec
CONSTANTS N = 3, Max = 5
INVARIANTS TypeInv, Safety
PROPERTIES Liveness
```

Ogni modulo TLA+ può avere più modelli di verifica, ciascuno con un proprio file di configurazione, così da consentire la validazione di scenari diversi senza modificare la specifica. Tra le opzioni più comuni vi sono il controllo dei deadlock, il numero di thread di esecuzione, il limite di profondità dell'esplorazione, la riduzione per simmetria e i semi casuali per la modalità di simulazione. Il log del controllo riporta il numero di stati generati, la profondità massima raggiunta, eventuali errori e le statistiche di copertura delle azioni.

`TLC` mette inoltre a disposizione operatori diagnostici definiti nel modulo omonimo, come `Assert` e `Print`, utili per verificare condizioni locali o osservare il valore delle variabili durante la verifica. Questi costrutti non modificano la semantica della specifica, ma consentono di eseguire controlli aggiuntivi durante l'analisi.

Nonostante l'efficienza delle sue ottimizzazioni, `TLC` resta un verificatore a stati espliciti: la dimensione del dominio delle costanti e la complessità delle strutture dati influenzano direttamente la crescita dello spazio esplorato. Per questo motivo, nella pratica, la modellazione in TLA+ richiede un bilanciamento tra fedeltà del modello e verificabilità, evitando domini troppo ampi e introducendo invarianti o astrazioni che ne riducano la complessità.

3.3 Apalache: verifica simbolica

`Apalache` è un model checker simbolico sviluppato per superare uno dei principali limiti di `TLC`, ossia l'esplosione combinatoria dello spazio degli stati. Invece di generare ed esplorare esplicitamente tutte le configurazioni possibili, `Apalache` traduce la specifica in un insieme di vincoli logici e ne delega la risoluzione a un *SMT solver* (*Satisfiability Modulo Theories*), come `Z3`. In questo modo è in grado di analizzare modelli con domini numerici o strutture di dati

molto più ampi, altrimenti impraticabili per un'esplorazione esaustiva.

Il funzionamento di `Apalache` si articola in più fasi. Anzitutto, la specifica TLA+ viene *normalizzata*, ossia riscritta in una forma intermedia più vicina alla logica dei vincoli. Successivamente, vengono generate formule che rappresentano simbolicamente gli stati e le transizioni del sistema. Tali vincoli sono quindi passati al solver, che ricerca un controesempio capace di violare le proprietà dichiarate, come un invariante o una condizione di correttezza.

A differenza di `TLC`, che esplora lo spazio degli stati in modo esaustivo, `Apalache` utilizza il metodo del *bounded model checking*. Questo approccio limita l'analisi a sequenze di lunghezza finita, verificando se entro un certo numero di passi esiste un comportamento che infrange le proprietà specificate. Pur non garantendo la correttezza assoluta oltre la profondità scelta, il metodo consente di individuare rapidamente errori concreti e di validare modelli complessi in tempi ridotti.

`Apalache` fornisce inoltre strumenti per l'analisi di tipi, la generazione di invarianti e la stima della copertura delle transizioni. Grazie all'analisi simbolica, risulta particolarmente adatto nelle fasi iniziali di sviluppo di una specifica, quando l'obiettivo è individuare anomalie logiche o incoerenze strutturali senza dover limitare eccessivamente il dominio dei parametri. In pratica, `Apalache` si affianca a `TLC`, unendo la completezza del controllo esplicito con l'efficienza del ragionamento simbolico.

3.4 TLAPS: sistema di dimostrazione formale

`TLAPS` (*TLA+ Proof System*) è il sistema di dimostrazione formale associato a TLA+, progettato per consentire la verifica deduttiva delle proprietà di una specifica. A differenza dei model checker, che esplorano o simulano lo spazio

degli stati, `TLAPS` fornisce una prova matematica della correttezza, basata su regole logiche e principi di deduzione formale.

Le prove vengono scritte direttamente all'interno dei moduli TLA+, utilizzando una sintassi dichiarativa che richiama la struttura dei teoremi matematici. Ogni dimostrazione è composta da una sequenza di passi logici, che possono essere verificati automaticamente o con l'intervento dell'utente. L'architettura di `TLAPS` è modulare: un componente di front-end gestisce l'analisi sintattica delle prove, mentre diversi back-end — tra cui `Zenon`, `Isabelle` e dimostratori interni — si occupano della verifica effettiva dei singoli passi.

Un uso comune consiste nel dimostrare che una certa proprietà è un invariante della specifica. In questi casi, la prova deve confermare che la proprietà vale nello stato iniziale (`Init`) e che continua a valere dopo ogni transizione descritta da `Next`. Una volta completata, la dimostrazione entra a far parte del modulo stesso, fornendo una garanzia formale della correttezza del sistema.

`TLAPS` permette di ottenere una *verifica formale totale*, in cui la validità della specifica non si basa su un'esplorazione automatica, ma su un ragionamento logico completo. L'integrazione tra `TLC`, `Apalache` e `TLAPS` rende possibile affrontare ogni livello della verifica: dal controllo automatico dei comportamenti concreti alla dimostrazione deduttiva delle proprietà essenziali.

Capitolo 4

Processo di specifica e verifica

Il processo di specifica e verifica formale rappresenta il cuore dell'approccio TLA+. Questo capitolo illustra il passaggio dalla definizione dei requisiti alla validazione logica del modello, seguendo le principali fasi operative del metodo.

Come mostrato in Figura 4.1, il percorso si articola in cinque passaggi fondamentali. Sebbene il diagramma presenti le fasi in modo lineare, nella pratica il processo è di natura iterativa: la verifica o il raffinamento possono richiedere di rivedere i requisiti o modificare la specifica, fino al raggiungimento di un modello stabile e coerente.

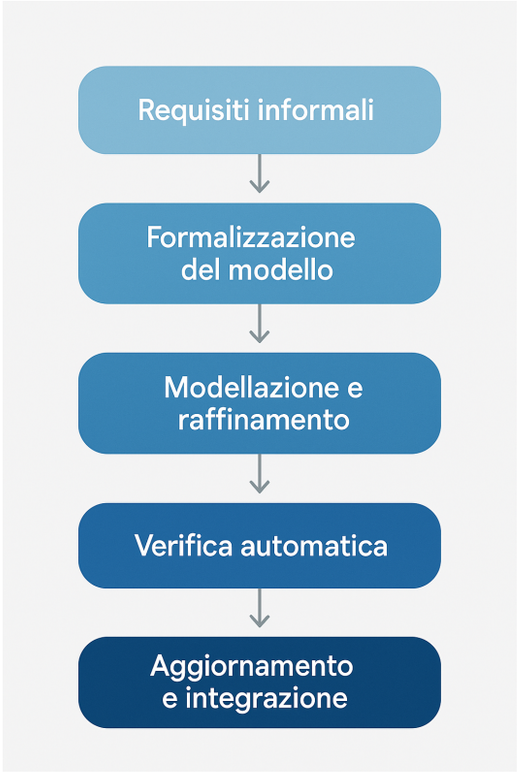


Figura 4.1: Fasi principali del processo di specifica e verifica formale.

Ciascuna fase viene ora analizzata in dettaglio, utilizzando come caso di riferimento un semplice sistema di prenotazione. I frammenti di codice TLA+ hanno valore esemplificativo e mostrano come i requisiti informali possano evolvere in una specifica formale verificabile e progressivamente raffinata.

4.1 Dalla descrizione informale alla specifica formale

Ogni processo di specifica formale parte da una descrizione informale del sistema da modellare. In questa fase vengono raccolti i requisiti del problema in forma testuale, tramite diagrammi UML o schemi concettuali, che descrivono

ciò che il sistema deve fare, ma senza la precisione necessaria per una verifica automatica. L'obiettivo del metodo formale è trasformare tali descrizioni qualitative in un modello matematico rigoroso, capace di rappresentare in modo non ambiguo i comportamenti del sistema.

Il sistema di prenotazione considerato presenta i seguenti requisiti principali:

- un utente può richiedere la prenotazione di una risorsa disponibile;
- una prenotazione diventa effettiva solo dopo conferma;
- una risorsa non può essere assegnata a più utenti contemporaneamente.

Il primo passo consiste nell'individuare le *entità rilevanti* e le corrispondenti *variabili di stato*. Nel modello sono presenti:

- `risorseDisponibili`: insieme delle risorse non ancora prenotate;
- `prenotazioni`: insieme delle richieste attive;
- `confermate`: insieme delle prenotazioni validate.

Si definiscono quindi le *azioni* che descrivono come cambia lo stato del sistema. Ogni azione rappresenta un evento che modifica una o più variabili del modello e viene espressa in TLA+ come una relazione logica tra lo stato corrente e quello successivo:

```
RichiediPrenotazione ==
    /\ risorsa \in risorseDisponibili
    /\ risorseDisponibili' = risorseDisponibili \ {risorsa}
    /\ prenotazioni' = prenotazioni \cup {risorsa}
    /\ UNCHANGED confermate
```

Le altre operazioni, come la conferma o l'annullamento della prenotazione, sono definite in modo analogo. Combinando tutte le azioni si costruisce la relazione di transizione complessiva del modello.

Infine, vengono introdotte le formule `Init` e `Next`, che definiscono rispettivamente lo stato iniziale del sistema e le sue regole di evoluzione:

- `Init` specifica la condizione di partenza, in cui tutte le risorse sono disponibili e non esistono prenotazioni attive;
- `Next` descrive le azioni che determinano i passaggi da uno stato all'altro.

Queste definizioni, prese insieme, costituiscono la formula `Spec`, ovvero la descrizione completa del comportamento del sistema:

```
Spec == Init /\ [][Next]_<<risorseDisponibili, prenotazioni,
    confermate>>
```

Questa formulazione è il punto di partenza per la verifica automatica. L'espressione dei requisiti in forma logica elimina le ambiguità del linguaggio naturale e consente di individuare incoerenze o omissioni prima dell'implementazione. La specifica formale diventa così una base condivisa tra progettisti e verificatori, garantendo una comprensione univoca delle regole del sistema.

4.2 Modellazione incrementale e raffinamento

La costruzione di una specifica TLA+ procede per passi successivi. Il modello evolve attraverso versioni progressivamente più dettagliate, ognuna delle quali introduce nuove condizioni o componenti per avvicinarsi al comportamento reale del sistema. Questo approccio, detto *modellazione incrementale*, consente di gestire la complessità e di individuare precocemente eventuali errori concettuali.

Nel caso del sistema di prenotazione, la prima versione può limitarsi a rappresentare l'aggiunta e la rimozione di prenotazioni, senza considerare eccezioni o conferme. Tale modello iniziale serve a verificare la coerenza delle regole fondamentali.

Successivamente vengono introdotti elementi aggiuntivi — come le prenotazioni confermate, i limiti di tempo o le politiche di annullamento — generando un *raffinamento* del modello precedente. In termini formali, una specifica S_1 raffina una specifica più astratta S_0 quando ogni comportamento ammesso da S_1 corrisponde a un comportamento ammesso da S_0, ovvero $\texttt{Beh}(S_1) \subseteq \texttt{Beh}(S_0)$. La nuova versione non introduce violazioni rispetto al modello di partenza, ma ne estende il comportamento in modo coerente.

Il raffinamento può essere:

- funzionale, quando vengono aggiunte nuove operazioni (ad esempio la conferma di una prenotazione);
- strutturale, quando si introducono variabili interne (come un contatore o una coda) senza modificare la semantica esterna del sistema.

Dopo ogni raffinamento il modello viene nuovamente verificato. Le proprietà già accertate — come invarianti o condizioni di vivacità — devono rimanere valide. Questo controllo iterativo garantisce che la complessità cresca in modo controllato e che ogni nuova versione sia logicamente fondata su quella precedente.

La modularità di TLA+ supporta naturalmente questo processo. Ogni componente può essere modellato come un modulo indipendente, verificato separatamente e poi integrato nella specifica complessiva. Ciò favorisce il riuso di parti già validate e semplifica l'analisi, concentrando l'attenzione su un numero limitato di variabili e azioni.

La modellazione incrementale e il raffinamento progressivo costituiscono quindi il nucleo metodologico della specifica formale. Partendo da un modello astratto e iterando la verifica dopo ogni estensione, si ottiene una descrizione sempre più precisa e aderente al comportamento reale del sistema, riducendo il rischio di errori e migliorando la qualità complessiva del progetto.

4.3 Verifica automatica del modello

Definita la specifica, il passo successivo consiste nel verificarne automaticamente la correttezza rispetto ai requisiti stabiliti. L'obiettivo non è solo individuare errori logici, ma anche confermare che il modello rispetti le proprietà di correttezza previste.

Nel caso del sistema di prenotazione, le principali proprietà da verificare sono:

- Invarianza: nessuna risorsa può trovarsi contemporaneamente negli insiemi `prenotazioni` e `confermate`;
- Sicurezza: una prenotazione confermata non può essere rimossa senza un'azione esplicita di rilascio;
- Vivacità: ogni richiesta deve, prima o poi, essere accettata o respinta.

La verifica di tali proprietà avviene tramite il `TLC`, descritto nel capitolo 3.2. Una volta definite le formule `Init`, `Next` e `Spec`, `TLC` esplora lo spazio degli stati generato dalla specifica e controlla automaticamente che gli invarianti e le proprietà temporali siano rispettati. Nel sistema di prenotazione, si considerano due invarianti principali:

- `TypeInv`, che garantisce la coerenza dei tipi e delle variabili di stato;

- `SafeAlloc`, che assicura che una risorsa non possa essere prenotata da più utenti contemporaneamente.

```
TypeInv == /\ prenotazioni \subseteq risorseDisponibili \cup
    confermate
SafeAlloc == \A r \in prenotazioni : r \notin confermate
```

Le configurazioni da analizzare sono determinate dai valori assegnati alle costanti nel file di configurazione del modello:

```
----------------- MODEL: SistemaPrenotazione -----------------
SPECIFICATION Spec
CONSTANTS Utenti = {u1, u2}, Risorse = {r1, r2}
INVARIANTS TypeInv, SafeAlloc
PROPERTIES Liveness
================================================================
```

Se una proprietà risulta violata, il verificatore produce una sequenza di stati che mostra come si è arrivati alla violazione. Tale informazione, visualizzabile nel Toolbox, consente di individuare quale azione o condizione ha generato il comportamento errato e di correggere la specifica o i requisiti corrispondenti.

La verifica automatica include anche il controllo di *deadlock*, cioè stati in cui nessuna azione è più abilitata. Nel modello di prenotazione, un deadlock può indicare che tutte le risorse sono assegnate e nessuna regola consente ulteriori modifiche di stato, suggerendo la necessità di rivedere la logica di rilascio delle risorse. Il rilevamento di un deadlock non implica necessariamente un errore nella specifica: può anche rappresentare una condizione terminale prevista dal modello. È quindi importante distinguere i blocchi legittimi, che indicano la conclusione del comportamento, da quelli indesiderati, in cui il sistema rimane bloccato pur avendo ancora azioni abilitate.

Oltre alla modalità esaustiva, `TLC` può operare in simulazione, generando comportamenti casuali per una validazione preliminare del modello senza esplorare l'intero spazio degli stati. Questo approccio è utile nelle fasi iniziali del progetto, quando l'obiettivo principale è confermare la coerenza generale delle formule.

La verifica automatica rappresenta quindi il momento di confronto tra la specifica formale e i requisiti progettuali: consente di validare le ipotesi, individuare incongruenze e fornire un riscontro oggettivo sulla correttezza del modello prima dell'implementazione.

4.4 Aggiornamento e integrazione

L'analisi dei risultati può portare alla necessità di rivedere i requisiti o di modificare la specifica, in un ciclo di perfezionamento continuo che costituisce uno degli aspetti distintivi del metodo TLA+.

Quando una proprietà risulta violata, la traccia generata dal verificatore mostra con precisione la sequenza di azioni che ha condotto all'errore. Tali informazioni permettono di identificare la causa logica della discrepanza e di intervenire in modo mirato: a volte è sufficiente correggere una formula o aggiungere un vincolo, altre è necessario riconsiderare un requisito mal definito o contraddittorio.

Il modello corretto viene quindi sottoposto a una nuova fase di verifica, fino al raggiungimento di una versione stabile che soddisfa tutti i vincoli di sicurezza e di vivacità. Una volta verificato, il modello TLA+ funge da riferimento per l'implementazione e per la fase di test, garantendo che il comportamento del sistema reale rispetti le regole formalmente definite.

In contesti industriali o di ricerca, la specifica verificata viene spesso man-

tenuta come documento vivo, aggiornato in parallelo al codice sorgente. In questo modo la coerenza tra progetto e implementazione rimane verificabile nel tempo, riducendo la possibilità di regressioni logiche e aumentando l'affidabilità complessiva del sistema.

L'ultima fase del processo non chiude il ciclo, ma lo rinnova: ogni modifica del sistema reale può tradursi in un aggiornamento del modello, che a sua volta può essere nuovamente verificato. La forza di TLA+ risiede proprio in questa capacità di coniugare rigore matematico e flessibilità progettuale, fornendo un metodo sistematico per mantenere la correttezza lungo l'intero ciclo di vita del software.

Capitolo 5

Casi di studio di TLA+

Questo capitolo analizza alcuni esempi significativi di applicazione di TLA+, al fine di mostrare come i principi introdotti nei capitoli precedenti trovino riscontro in contesti pratici di progettazione e verifica. L'attenzione è rivolta in particolare ai sistemi distribuiti, ambito in cui la complessità delle interazioni e la possibilità di comportamenti concorrenti rendono i metodi formali uno strumento di analisi particolarmente efficace.

Nei sistemi di questo tipo, diversi processi cooperano per raggiungere un obiettivo comune, scambiandosi informazioni attraverso canali di comunicazione potenzialmente inaffidabili. Guasti parziali, ritardi di rete o perdite di messaggi possono generare situazioni difficili da riprodurre mediante test empirici, ma che possono essere esplorate in modo esaustivo con la modellazione formale.

Per introdurre l'approccio, verrà presentato innanzitutto un modello classico, il protocollo *Two-Phase Commit*, utilizzato per garantire l'atomicità delle transazioni distribuite. Successivamente verrà analizzato un caso reale di impiego industriale: l'adozione di TLA+ da parte di Amazon Web Services per la verifica dei protocolli di replica nel servizio *S3*.

5.1 Il protocollo Two-Phase Commit

Il protocollo Two-Phase Commit (2PC) rappresenta uno dei meccanismi più utilizzati per garantire l'atomicità delle transazioni distribuite. È impiegato in numerosi sistemi di database e servizi cloud per assicurare che un gruppo di nodi partecipanti esegua un'operazione in modo coerente: o tutti confermano la transazione, oppure tutti la annullano.

Nel contesto dei sistemi distribuiti, tale comportamento è essenziale per evitare stati di inconsistenza dovuti a guasti o ritardi di comunicazione. L'approccio formale offerto da TLA+ consente di modellare in modo rigoroso la sequenza di stati e di azioni che caratterizzano il protocollo, rendendo possibile la verifica automatica delle sue proprietà di correttezza.

Il funzionamento del 2PC coinvolge due ruoli distinti: un coordinatore, che avvia la transazione e raccoglie le decisioni dei partecipanti, e uno o più partecipanti, che eseguono l'operazione locale e comunicano la propria disponibilità a completarla. L'esecuzione è articolata in due fasi principali. Nella prima, detta fase di preparazione, il coordinatore invia un messaggio di richiesta (`PREPARE`) e i partecipanti rispondono con `VOTE_COMMIT` o `VOTE_ABORT`. Nella seconda, la fase di commit, se tutti i partecipanti hanno risposto positivamente, il coordinatore invia `GLOBAL_COMMIT`; in caso contrario invia `GLOBAL_ABORT`.

Il frammento seguente mostra un modello semplificato del protocollo espresso in linguaggio TLA+.

```
---------------------- MODULE TwoPhaseCommit ----------------------
EXTENDS TLC

CONSTANT Participants
VARIABLES state, decision
```

```
Init ==
    /\ state = [p \in Participants |-> "init"]
    /\ decision = "undecided"

Prepare(p) ==
    /\ state[p] = "init"
    /\ state' = [state EXCEPT ![p] = "prepared"]
    /\ UNCHANGED decision

VoteAbort(p) ==
    /\ state[p] = "init"
    /\ state' = [state EXCEPT ![p] = "aborted"]
    /\ UNCHANGED decision

Commit ==
    /\ \A p \in Participants : state[p] = "prepared"
    /\ decision' = "commit"
    /\ UNCHANGED state

Abort ==
    /\ \E p \in Participants : state[p] = "aborted"
    /\ decision' = "abort"
    /\ UNCHANGED state

Next ==
    \/ \E p \in Participants : Prepare(p)
    \/ \E p \in Participants : VoteAbort(p)
    \/ Commit
    \/ Abort
```

```
Spec == Init /\ [][Next]_<<state, decision>>

NoPartialCommit ==
    ~(\E p1, p2 \in Participants :
        state[p1] = "prepared" /\ state[p2] = "aborted")
=============================================================================
```

Nel modello sono dichiarate due variabili principali: `state`, che rappresenta lo stato di ciascun partecipante, e `decision`, che indica la decisione globale del coordinatore. La formula `Init` definisce la configurazione iniziale del sistema, in cui tutti i partecipanti si trovano nello stato `"init"` e nessuna decisione è ancora stata presa.

Le azioni `Prepare(p)` e `VoteAbort(p)` rappresentano le risposte locali dei partecipanti alla richiesta di commit. Ogni partecipante può passare dallo stato iniziale a quello di "preparato" oppure decidere di abortire; in entrambi i casi, la variabile `decision` rimane invariata, poiché il coordinatore non ha ancora deliberato. Le azioni `Commit` e `Abort` descrivono invece la decisione globale: nel primo caso tutti i partecipanti devono trovarsi nello stato `"prepared"`, nel secondo è sufficiente che uno abbia dichiarato `"aborted"`. L'operatore `Next` raccoglie tutte le transizioni possibili, mentre la formula `Spec` unisce la condizione iniziale alla dinamica temporale del sistema.

Infine, la formula `NoPartialCommit` rappresenta un'invariante di sicurezza: il sistema non deve mai raggiungere uno stato in cui alcuni partecipanti abbiano votato per il commit e altri per l'abort.

Per verificare il modello è stato utilizzato il *TLA+ Toolbox*, impostando due partecipanti `P1` e `P2` e selezionando l'invariante `NoPartialCommit` come proprietà da controllare. Il risultato dell'esecuzione è riportato in Figura 5.1.

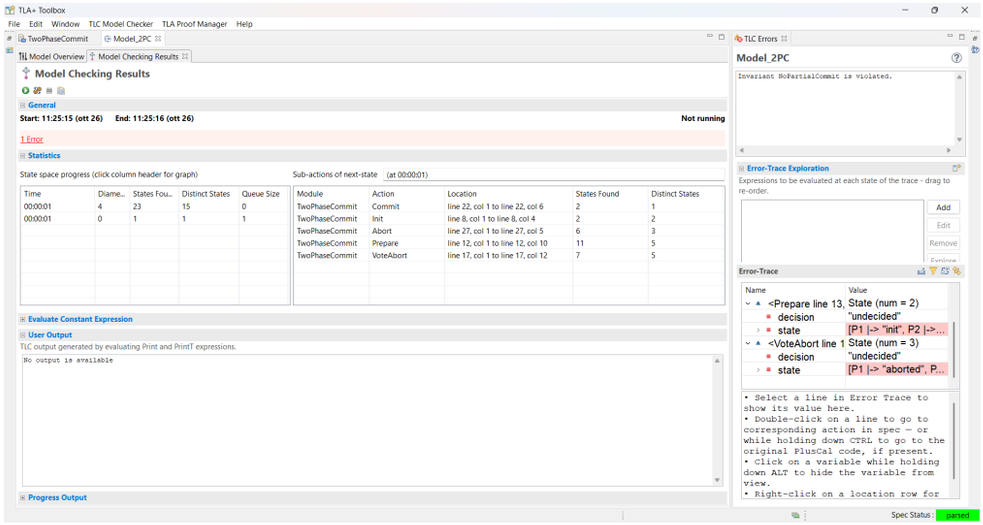


Figura 5.1: Violazione dell'invariante `NoPartialCommit` nel modello Two-Phase Commit.

Il model checker ha rilevato la violazione dell'invariante, segnalando uno stato in cui un partecipante risulta `"prepared"` mentre un altro si trova nello stato `"aborted"`. Questa condizione rappresenta una configurazione incoerente rispetto all'obiettivo del protocollo, che dovrebbe garantire una decisione globale univoca. L'anomalia non è dovuta a un errore del linguaggio o dello strumento, ma alla semplificazione introdotta nel modello: nel frammento proposto non è presente un vero coordinatore che gestisca l'ordine delle operazioni. Le azioni dei partecipanti possono quindi avvenire in parallelo, generando stati transitori non sincronizzati.

```
------------------ MODULE TwoPhaseCommit_Corrected ------------------
EXTENDS TLC

CONSTANT Participants
VARIABLES pstate, coordDecision
```

```
Init ==
    /\ pstate = [p \in Participants |-> "init"]
    /\ coordDecision = "undecided"

Prepare(p) ==
    /\ coordDecision = "undecided"
    /\ pstate[p] = "init"
    /\ pstate' = [pstate EXCEPT ![p] = "prepared"]
    /\ UNCHANGED coordDecision

VoteAbort(p) ==
    /\ coordDecision = "undecided"
    /\ pstate[p] = "init"
    /\ pstate' = [pstate EXCEPT ![p] = "aborted"]
    /\ UNCHANGED coordDecision

CollectCommit ==
    /\ coordDecision = "undecided"
    /\ \A q \in Participants : pstate[q] = "prepared"
    /\ coordDecision' = "commit"
    /\ UNCHANGED pstate

CollectAbort ==
    /\ coordDecision = "undecided"
    /\ \E q \in Participants : pstate[q] = "aborted"
    /\ coordDecision' = "abort"
    /\ UNCHANGED pstate

ApplyCommit(p) ==
```

```
    /\ coordDecision = "commit"
    /\ pstate[p] = "prepared"
    /\ pstate' = [pstate EXCEPT ![p] = "committed"]
    /\ UNCHANGED coordDecision

ApplyAbort(p) ==
    /\ coordDecision = "abort"
    /\ pstate[p] # "aborted"
    /\ pstate' = [pstate EXCEPT ![p] = "aborted"]
    /\ UNCHANGED coordDecision

Next ==
    \/ \E p \in Participants : Prepare(p)
    \/ \E p \in Participants : VoteAbort(p)
    \/ CollectCommit
    \/ CollectAbort
    \/ \E p \in Participants : ApplyCommit(p)
    \/ \E p \in Participants : ApplyAbort(p)

Spec == Init /\ [][Next]_<<pstate, coordDecision>>

NoCommitWithAbort ==
    ~(\E p,q \in Participants : pstate[p] = "committed" /\ pstate[q]
       = "aborted")
=============================================================================
```

Il nuovo modello introduce la variabile `coordDecision`, che rappresenta la decisione globale del coordinatore, e separa le fasi di decisione (`CollectCommit`, `CollectAbort`) da quelle di applicazione (`ApplyCommit`, `ApplyAbort`). In que-

sto modo il coordinatore raccoglie i voti e stabilisce il risultato prima che le modifiche vengano propagate ai partecipanti, impedendo che gli stati locali divergano.

La verifica condotta nel *TLA+ Toolbox*, con due partecipanti e l'invariante `NoCommitWithAbort`, ha confermato che tutte le proprietà di sicurezza sono soddisfatte e nessuna violazione si verifica durante l'esplorazione dello spazio degli stati (Figura 5.2).

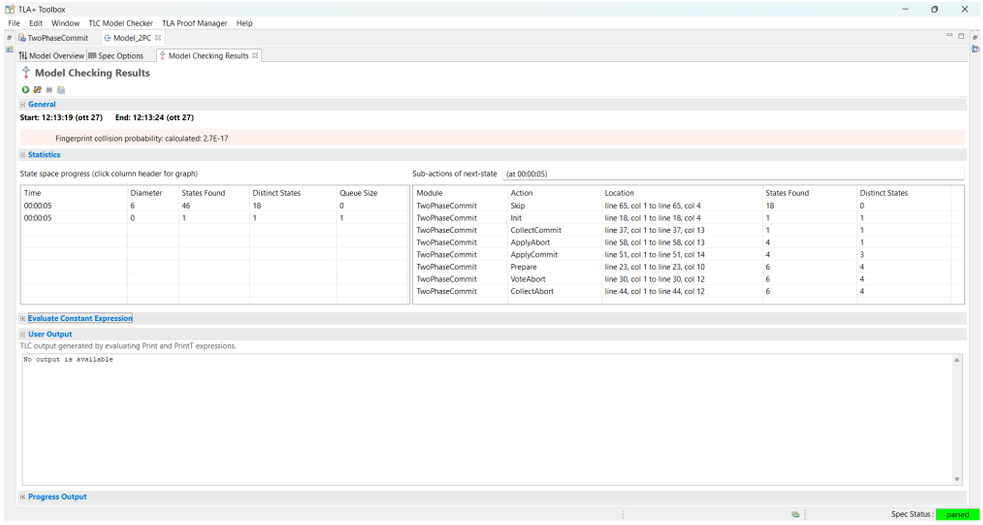


Figura 5.2: Verifica del modello corretto del protocollo Two-Phase Commit: tutte le invarianti soddisfatte.

L'introduzione del coordinatore e delle regole di sincronizzazione risolve la condizione di incoerenza riscontrata nel modello iniziale. Il confronto tra le due versioni mostra come l'uso di TLA+ consenta non solo di verificare la correttezza del sistema, ma anche di riconoscere in modo metodico le origini di un malfunzionamento e di confermare, tramite successive revisioni del modello, l'efficacia delle correzioni apportate.

5.2 Amazon S3

Amazon S3 (Simple Storage Service) è il servizio di archiviazione oggetti di Amazon Web Services, progettato per offrire affidabilità e disponibilità a livello globale. I dati vengono replicati in più data center, così da garantire persistenza anche in caso di guasti. Il sistema deve assicurare che ogni operazione di scrittura, cancellazione o replica mantenga la coerenza tra nodi distribuiti in regioni diverse, nonostante ritardi, errori di rete o riavvii imprevisti.

In un'infrastruttura di queste dimensioni, la complessità non risiede solo nel codice, ma soprattutto nei protocolli di comunicazione tra componenti. Per questo motivo, a partire dal 2011, i team di ingegneri di AWS hanno introdotto l'uso sistematico di TLA+ per modellare e verificare i protocolli di S3 e di altri servizi critici. L'obiettivo non era tradurre il software in logica matematica, ma creare modelli astratti capaci di descrivere il comportamento globale del sistema e individuare situazioni anomale impossibili da riprodurre con i test convenzionali.

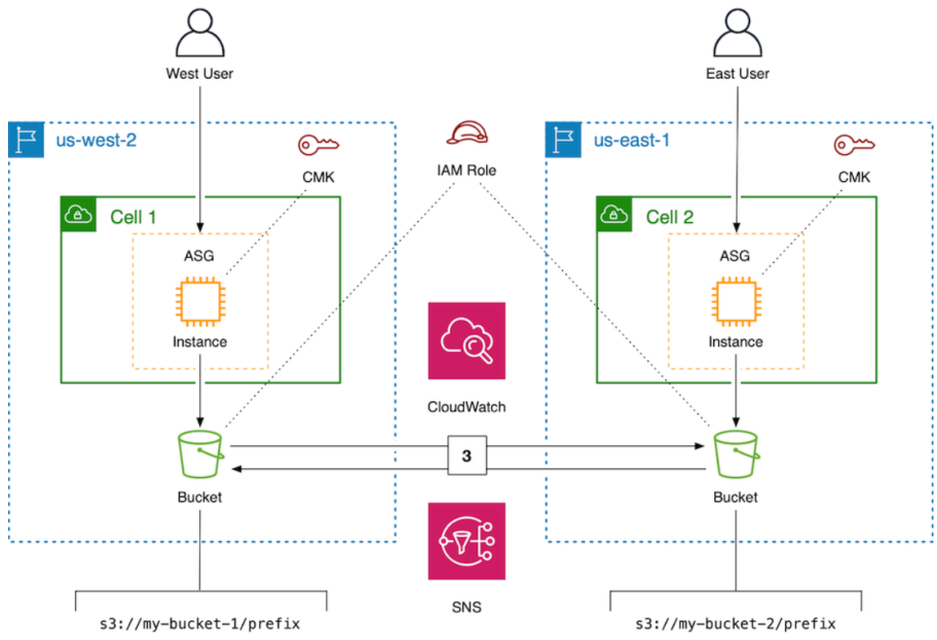


Figura 5.3: Schema semplificato della replica tra regioni in Amazon S3. Le modifiche a un oggetto vengono propagate tra bucket distinti per garantire durabilità e consistenza. Servizi come CloudWatch e SNS supportano il processo di replica tramite monitoraggio e notifiche.

Attraverso i modelli formali, gli ingegneri di Amazon hanno verificato le proprietà di sicurezza e vivacità dei protocolli di replica: che nessun dato possa andare perso anche in presenza di fallimenti, e che ogni scrittura venga infine propagata a tutte le repliche. Durante l'analisi di S3, l'uso di TLA+ ha permesso di scoprire errori logici nel design che non erano mai emersi nei test automatici, spesso dovuti a interazioni tra più componenti asincroni. Molti di questi errori logici avrebbero potuto causare inconsistenze silenziose o perdita temporanea di dati.

Il principio verificato con TLA+ è analogo a quello di un quorum di commit: un'operazione è considerata completata solo quando un numero minimo di repliche ha confermato la scrittura. La seguente specifica, pur semplificata, illustra il tipo di modello astratto usato per ragionare sulla correttezza del

protocollo.

```
---------------------- MODULE S3Replication --------------------
EXTENDS TLC

CONSTANT Replicas
VARIABLES state, commit

Init ==
    /\ state = [r \in Replicas |-> "idle"]
    /\ commit = "pending"

Write(r) ==
    /\ state[r] = "idle"
    /\ state' = [state EXCEPT ![r] = "written"]
    /\ UNCHANGED commit

Ack(r) ==
    /\ state[r] = "written"
    /\ state' = [state EXCEPT ![r] = "acked"]
    /\ UNCHANGED commit

Commit ==
    /\ Cardinality({r \in Replicas : state[r] = "acked"}) >= 2
    /\ commit' = "done"
    /\ UNCHANGED state

Next ==
    \/ \E r \in Replicas : Write(r)
    \/ \E r \in Replicas : Ack(r)
```

```
    \/ Commit

Spec == Init /\ [][Next]_<<state, commit>>

Safety ==
    ~(\E r \in Replicas : state[r] = "acked" /\ commit = "pending")
=============================================================
```

Il modello rappresenta un insieme di repliche che ricevono scritture (`Write`) e inviano conferme (`Ack`). La condizione di `Commit` garantisce che l'operazione venga finalizzata solo dopo un numero sufficiente di conferme, simulando il comportamento di un quorum. L'invariante `Safety` assicura che non esistano stati intermedi incoerenti, nei quali una replica abbia completato la scrittura mentre il sistema nel complesso non ha ancora deciso il commit.

Amazon ha riportato che, grazie all'uso di TLA+, sono stati individuati e corretti numerosi difetti di progettazione nei protocolli di S3, DynamoDB ed EBS, prima ancora della fase di implementazione. L'adozione della verifica formale è poi diventata parte integrante del ciclo di sviluppo dei servizi AWS, permettendo ai team di ingegneria di validare rapidamente nuove modifiche e di individuare regressioni logiche prima che raggiungano il codice di produzione.

5.3 Altri casi di adozione industriale

L'esperienza di Amazon non rappresenta un caso isolato, ma fa parte di una tendenza più ampia che ha portato diversi attori del settore tecnologico ad adottare TLA+ come strumento di verifica formale. La necessità di garantire coerenza e affidabilità in architetture sempre più distribuite ha spinto numerose aziende a integrare la modellazione logica nei propri processi di progettazione.

Microsoft, ad esempio, ha impiegato TLA+ per verificare i protocolli di sincronizzazione e i sistemi di replica del servizio Azure Storage, individuando scenari di concorrenza che i test tradizionali non riuscivano a rilevare. Intel lo ha utilizzato per analizzare i protocolli di coerenza della cache nei processori multicore, assicurando che i diversi core condividessero correttamente lo stato della memoria. Elastic, Dropbox e Oracle lo hanno applicato alla verifica degli algoritmi di consenso e dei meccanismi di replicazione nei propri sistemi di archiviazione distribuita.

In molti di questi contesti, i modelli TLA+ vengono mantenuti come parte della documentazione tecnica, aggiornati a ogni revisione architetturale e utilizzati per validare modifiche o nuove funzionalità. La specifica formale diventa così una fonte di verità condivisa, capace di descrivere in modo verificabile il comportamento del sistema.

Questa diffusione industriale conferma che l'adozione dei metodi formali non è più confinata all'ambito accademico, ma costituisce una pratica ingegneristica matura. L'esperienza di aziende come Amazon, Microsoft e Intel dimostra che la verifica matematica non rallenta lo sviluppo, ma contribuisce a ridurre i costi di manutenzione e ad aumentare la sicurezza e la stabilità delle infrastrutture digitali.

Conclusioni

Questo lavoro ha esplorato il valore della verifica formale come fondamento della progettazione di sistemi software affidabili. Attraverso l'analisi del linguaggio TLA+ e dei casi di studio affrontati, è emerso come la modellazione matematica consenta di individuare errori logici e di definire con precisione le proprietà che un sistema deve rispettare per essere considerato corretto. In un contesto in cui il software governa infrastrutture critiche e servizi distribuiti su scala globale, la capacità di prevedere il comportamento del sistema prima della sua realizzazione non è solo un vantaggio tecnico, ma una necessità.

L'uso dei metodi formali consente di passare da un insieme di ipotesi operative a un progetto che può essere controllato e verificato in modo oggettivo. Modellare significa rendere esplicite le assunzioni, i vincoli e le condizioni di fallimento, riducendo la dipendenza dall'intuizione e dall'esperienza empirica. Il linguaggio TLA+ si è rivelato uno strumento concreto per collegare teoria e pratica, consentendo di descrivere protocolli reali e verificarne le proprietà fondamentali di sicurezza e progresso. Questa integrazione tra rigore logico e progettazione ingegneristica rappresenta una nuova forma di maturità nello sviluppo del software.

Questo tema resta aperto. Oggi la digitalizzazione tocca ogni ambito della società: comunicazioni, finanza, sanità e perfino la sfera privata. In questo scenario, la sicurezza informatica non può più essere considerata un livello

aggiuntivo o un controllo a posteriori. È la condizione di base per la sopravvivenza dei sistemi digitali, il prerequisito di ogni innovazione. La complessità crescente dei sistemi distribuiti e l'interconnessione globale rendono inevitabili nuovi punti di vulnerabilità, e solo un approccio capace di dimostrare formalmente il corretto funzionamento dei meccanismi critici può garantire stabilità nel tempo.

Il percorso descritto in questa tesi evidenzia come la sicurezza non sia una proprietà accessoria del software, ma un principio costitutivo che nasce da una progettazione consapevole. In questo senso, la verifica formale può essere vista come un modo per costruire tecnologia più trasparente, controllabile e affidabile. Non significa soltanto produrre codice privo di errori, ma realizzare sistemi che incarnino la fiducia necessaria a sostenere una società sempre più dipendente dal digitale. Costruire sistemi corretti significa, in ultima analisi, costruire fiducia. Nel mondo digitale che ci attende, la fiducia sarà la risorsa più preziosa.

Bibliografia

[1] Lamport, *TLA+ Home*.

`https://lamport.azurewebsites.net/tla/tla.html`

[2] Lamport, *Specifying Systems*.

`https://lamport.azurewebsites.net/tla/book.html`

[3] Lamport, *The TLA+ Hyperbook*.

`https://lamport.azurewebsites.net/tla/hyperbook.html`

[4] Lamport, *The TLA+ Toolbox*.

`https://lamport.azurewebsites.net/tla/toolbox.html`

[5] Lamport, *TLA+ Tools*.

`https://lamport.azurewebsites.net/tla/tools.html`

[6] Lamport, *The PlusCal Tutorial*.

`https://lamport.azurewebsites.net/tla/tutorial/home.html`

[7] Lamport, Yu, *TLC Model Checker*.

`https://lamport.azurewebsites.net/tla/tlc.html`

[8] Lamport, *TLA+ Video Course*.

`https://lamport.azurewebsites.net/video/videos.html`

[9] Amazon, *Amazon S3*.

`https://aws.amazon.com/it/s3/`

[10] Wikipedia, *Formal Methods*.

`https://en.wikipedia.org/wiki/Formal_methods`

[11] Wikipedia, *Model Checking*.

`https://en.wikipedia.org/wiki/Model_checking`

[12] Wikipedia, *Two-Phase Commit*.

`https://en.wikipedia.org/wiki/Two-phase_commit_protocol`

[13] Wikipedia, *TLA+*.

`https://en.wikipedia.org/wiki/TLA+`

[14] Wikipedia, *Leslie Lamport*.

`https://en.wikipedia.org/wiki/Leslie_Lamport`

[15] Wikipedia, *Edsger W. Dijkstra*.

`https://en.wikipedia.org/wiki/Edsger_W._Dijkstra`

[16] Wikipedia, *Tony Hoare*.

`https://en.wikipedia.org/wiki/Tony_Hoare`